v5.0 — MULTI-AGENT SUPERINTELLIGENCE

TRINNITY VISERON SYSTEM

Sistema Operacional Multi-Agente de Superinteligenica

Portuguese · English · Española

%Æ5,000+ Autonomous AI Agents

%ÆDesktop WebOS Interface

%ÆMultilingual Voice Bridge

%Æn8n Automation Engine

%ÆSelf-Evolving Superintelligence

Prepared for: Startup Presentation · July 2026

Pedro Costa · Trinnity Hurtado

Trinnity Viseron System (TVS) v5.0 is a multi-agent artificial superintelligence operating system designed to operate autonomously, evolve continuously, and coordinate thousands of specialized AI agents toward complex goals. Unlike traditional AI platforms, TVS is not a chatbot or a single model — it is a self-organizing digital civilization.

The system features a full desktop WebOS interface accessible from any browser, a multilingual voice bridge (Portuguese, English, Spanish), an integrated n8n workflow automation engine, a token economy with $TRIN and $VSR tokens, and a hierarchical command structure inspired by military battalion organization. TVS can spawn and coordinate over 5,000 agents, each with specialized capabilities, across multiple squads and lineages.

Key Metrics

5,126
Autonomous Agents

246
Archetypes Loaded

32,070
Total Capabilities

98.92%
Average Wisdom

5
n8n Workflow Templates

3
Languages Supported

290+
AI Providers Connected

A4
WebOS Desktop Environment

Core Components

Core Engine

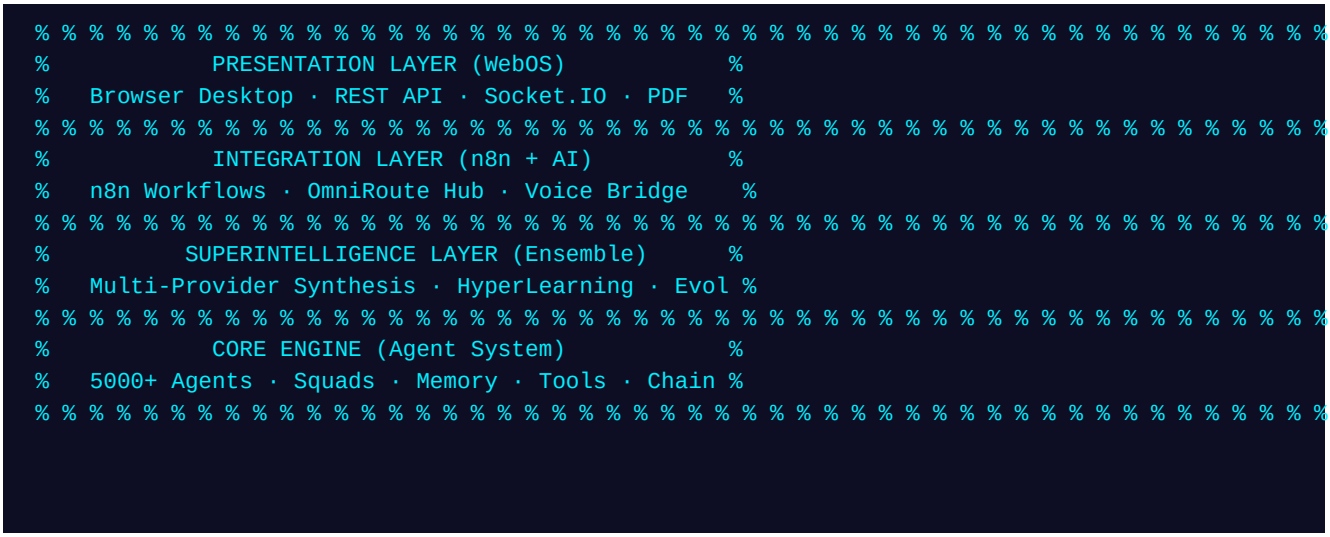
- ViseronCore — System orchestrator
- AgentManager — Agent lifecycle
- SquadManager — Team coordination
- ToolManager — Plugin & tool registry
- MemoryEngine — Short/long-term memory

Integration Layer

- OmniRoute Hub: 290+ AI providers
- n8n Bridge: 5 automation workflows
- OpenJarvis: Local Stanford AI
- Call System: Twilio + voice AI
- ASNO: WhatsApp + Home Assistant

Architecture Flow

TVS follows a layered architecture: Core Engine (agent ma
Layer (multi-provider synthesis, ensemble reasoning) !' In
systems) !' Presentation Layer (WebOS dashboard, REST A
a central event bus and command chain.



WebOS transforms the browser into a full desktop operating system experience. Users interact with a complete graphical environment featuring a taskbar, start menu, desktop icons, draggable windows, system tray, clock, and language selector. All built with vanilla JavaScript — no external framework required.

Built-in Applications

Ø=ÜÌ Terminal	Command-line interface to the Viseron system. Supports commands: help, status, agents, clear, voice, plan, token, lang, time
Ø=ÜÌ Monitor	Real-time system metrics display: total agents, minds loaded, intelligence level, evolution cycles, capabilities count
Ø=ÜÌ Agents	Browse all registered agents with status, role, and capabilities. Shows up to 50 agents at a time
Ø=ÜÌ Voice Control	Interface to the JARVIS Voice Bridge. Select speaker (Pedro/Trinnity) and send voice commands
Ø=ÜÌ Token Dashboard	View token economy: 300M $VSR and 1B $TRIN with allocation breakdown
Ø=ÜÌ Automation	n8n workflow engine UI. List and trigger 5 workflow templates directly from the desktop

Desktop Features

- %Ï Window Manager — Drag, resize, minimize, close, and focus windows with z-index stacking
- %Ï Start Menu — Search and launch applications with live filtering
- %Ï System Tray — Clock display with locale-aware formatting (PT/EN/ES)
- %Ï Language Switcher — Toggle between Portuguese, English, and Spanish instantly
- %Ï Three.js Background — Animated 3D particle system with torus knots and mouse parallax
- %Ï Responsive — Adapts to any screen size, mobile-friendly

TVS v5.0 is fully internationalized with three languages: Portuguese (Brazil), English (US), and Spanish. Every interface element — the WebOS desktop, all apps, the voice widget, system messages, and error reports — adapts to the selected language in real time without page reload.

Coverage

WebOS Desktop

- Start menu & taskbar labels
- Window titles and button text
- App descriptions and placeholders
- Clock format locale-aware
- Drag-and-drop tooltips

JARVIS Voice

- Status messages (idle, processing, responding)
- Microphone button labels
- Speaker selection (Pedro/Trinnity)
- All voice responses
- Fallback/error messages

System Dashboard

- Dashboard index.html title
- All API response descriptions
- System status indicators
- Agent role descriptions
- n8n workflow UI labels

Future Languages

- French (fr)
- German (de)
- Italian (it)
- Japanese (ja)
- Chinese (zh)
- Arabic (ar)

Architecture

Each component maintains its own language dictionary object (L) with keys for every translatable string. A helper function `t(key)` looks up the current language and returns the translated value. Language is detected from navigator.language on first load and can be toggled at runtime. The WebOS app registry also stores title keys that resolve through the same translation system.

The Voice Bridge is a real-time voice interaction system named JARVIS. It provides speech recognition, AI-powered response generation, and bidirectional communication via Socket.IO. Users can select between two speakers — Pedro (Commander) and Trinnity (Queen) — each with distinct response personalities.

Features

- %ï Speech-to-Text via Web Speech API (Chrome, Edge, Safari)
- %ï Socket.IO real-time communication with the backend
- %ï REST API fallback when WebSocket is unavailable
- %ï Two speaker profiles: Pedro (formal, military-style) and Trinnity (queenly, poetic)
- %ï Voice history tracking with clear endpoint
- %ï Voice command processing routes through AI agent system
- %ï Multilingual responses in PT, EN, ES based on selected language

API Endpoints

`POST /api/voice/command`	Send a voice command with { text, speaker }
`GET /api/voice/history`	Retrieve voice command history
`POST /api/voice/clear`	Clear voice history

Socket.IO Events

`voice:command`	Send voice command from client
`voice:response`	Receive AI-processed response
`voice:error`	Receive error messages
`voice:transcript`	Broadcast transcript to all clients

TVS integrates n8n as its automation and workflow engine. The N8NBridge provides a local workflow execution engine that can run template-based workflows with steps including webhooks, AI processing, tool execution, code execution, conditions, transformations, delays, and notifications. It attempts to start a real n8n process on port 5678 and falls back to the local engine if n8n is not available.

Workflow Templates

Process Voice Command	Webhook + AI + Code + Tool + Notification
Auto-Generate Report	Webhook + AI + Condition + Tool
Trigger Auto-Evolution	Webhook + AI + Code + Notification
Deploy Service	Webhook + Condition + Tool + Delay
	Webhook + Code + Tool + Notification

Creates new agents from voice commands or API requests

Routes voice commands through AI and executes actions

Generates PDF reports on schedule

Checks metrics and triggers evolution cycles every 5 min

Deploys any service via webhook and Docker

API Endpoints

`GET /api/workflows`	List all workflow templates with triggers
`POST /api/workflows/run`	Execute a workflow by ID with { workflowId, data }

TVS operates 5,000+ autonomous AI agents organized in a hierarchical structure inspired by military battalion organization. Agents are grouped into lineages (Corona and Hierro), squads, and areas. Each agent has a unique identity, role, capabilities, and execution engine.

Agent Hierarchy

Squad	Members	Function
Executive Squad	Pedro (Commander) + Trinnity (Queen)	Strategic leadership, high-level directives
Architecture Squad	Architect Prime + Dev Master + CyberSentinel	System design, development, security
Sovereigns	Top-tier agents with epithets (e.g., 'All-Seeing')	Autonomous decision-making
Corona Lineage	Commanders + specialists	Strategic command structure
Hierro Lineage	Operational commanders + specialists	Tactical execution
Archetypes	246 pre-defined agent archetypes	Specialized capability profiles

Battalion Registry API

`GET /api/battalion`	Full battalion statistics with lineage breakdown
`GET /api/battalion/:id`	Get specific agent by ID
`GET /api/agents`	List all registered agents
`GET /api/stats`	System intelligence metrics (wisdom, evolutions, etc.)

The command chain is the executive backbone of TVS. It issues strategic, architectural, and tactical directives that propagate through the squad hierarchy. Directives are stored, tracked, and executed by the DirectiveEngine with status monitoring.

Directive Types

Strategic Directives

- Issued by Commander Pedro
- System-wide strategic goals
- Superintelligence activation
- Long-term evolution targets
- Example: 'Activate 1000% intelligence'

Architectural Directives

- Issued by Queen Trinnity
- System architecture evolution
- Genetic evolution parameters
- Agent capability upgrades
- Example: '500% evolution every 30 min'

API Endpoints

`GET /api/directives`	List all active directives with stats
`POST /api/directive`	Issue a new directive with body payload
`GET /api/status`	Full system status with squads and stats

TVS features a built-in token generation engine that creates and manages digital tokens with full tokenomics. Two tokens are active: $TRIN (Trinnity) and $VSR (Viseron Crown). The token system includes supply management, allocation, and distribution tracking.

$TRIN — Trinnity Token

Token Details

- Symbol: $TRIN
- Total Supply: 1,000,000,000
- Type: Utility + Governance
- Purpose: AI agent resource allocation

$VSR — Viseron Crown

- Symbol: $VSR
- Total Supply: 300,000,000
- Type: Proof of Mandate (PoM)
- Purpose: Battalion command token

$VSR Allocation

Holder	Amount	Percentage
Trinnity	90,000,000	30%
Pedro	75,000,000	25%
Legion	90,000,000	30%
Reserve	45,000,000	15%

The dashboard server (TVSDashboardServer) runs on port 3000 and serves the WebOS interface, REST API endpoints, Socket.IO for real-time communication, and PDF report generation. It provides complete programmatic access to every system feature.

Complete API Reference

Endpoint	Description
`GET /api/health`	System health check
`GET /api/stats`	Intelligence metrics (agents, wisdom, evolutions)
`GET /api/agents`	List all agents
`GET /api/status`	Full system status with squads
`GET /api/battalion`	Battalion structure and statistics
`GET /api/battalion/:id`	Specific agent by ID
`GET /api/directives`	Active directives
`POST /api/directive`	Issue new directive
`POST /api/synthesize`	AI synthesis with { prompt }
`POST /api/voice/command`	Send voice command
`GET /api/voice/history`	Voice command history
`POST /api/voice/clear`	Clear voice history
`GET /api/workflows`	List n8n workflow templates
`POST /api/workflows/run`	Execute a workflow by ID
`GET /api/report/pdf`	Download system report PDF

Installation

`npm install`	Install all dependencies
`npm run build`	Compile TypeScript to dist/
`npm start`	Run the full system on port 3000
`npm run dev`	Development mode with hot reload (tsx)

Build Commands

`npm run build`	TypeScript compilation + copy static assets
`npm run build:android`	Build APK for Google Play
`npm run build:ios`	Build IPA for Apple Store (macOS)
`npm run build:all`	Build both Android + iOS
`npm run build:exe`	Build standalone executable
`npm run build:electron`	Build Electron desktop app

Mobile

`npm run mobile:start`	Start Expo dev server
`npm run mobile:android`	Run on Android device/emulator
`npm run mobile:ios`	Run on iOS simulator

Testing & Quality

`npm test`	Run core system tests
`npm run test:hyper`	Run hyperbrain tests
`npm run lint`	TypeScript type checking

WebOS Terminal Commands

`help or ?`	Show available commands
`status or stats`	Display system metrics
`agents or agent`	List registered agents
`clear`	Clear terminal output
`say <text>`	Send a voice command via JARVIS
`voice`	Open voice control interface
`plan`	Show current evolution plan
`token`	Display token information
`lang`	Toggle language (PT/EN/ES)
`time`	Show current system time

Local Development

```
git clone <repo>
cd Trinnity-Viseron-System
npm install
npm run build
npm start
# Open http://localhost:3000
```

Environment Variables (.env)

```
PORT=3000                      # Dashboard server port
OPENAI_API_KEY=sk-...          # OpenAI (optional)
ANTHROPIC_API_KEY=sk-ant-...   # Anthropic (optional)
GEMINI_API_KEY=...             # Google Gemini (optional)
XAI_API_KEY=...                # xAI Grok (optional)
# If no keys set, system uses Ollama (local)
```

Production Deployment

- %ï Build: `npm run build` compiles to dist/ and copies static assets
- %ï Run: `npm start` starts the production server
- %ï Standalone: `npm run build:exe` creates a single executable with pkg
- %ï Electron: `npm run build:electron` packages as a desktop app
- %ï Docker: n8n runs in container via docker-compose with Ollama + Qdrant

Vercel / Cloud Deployment

The WebOS dashboard is fully static and can be deployed to Vercel, Netlify, or any CDN. The backend requires a Node.js server (Express + Socket.IO). For cloud deployment, use Railway, Render, Fly.io, or a VPS. The mobile app builds via Expo EAS for both Android (APK/AAB) and iOS (IPA) app stores.

Current (v5.0)

- %ï WebOS desktop interface with 6 built-in apps
- %ï Multilingual support: PT, EN, ES
- %ï JARVIS Voice Bridge with speaker profiles
- %ï n8n workflow engine with 5 templates
- %ï 5,000+ autonomous AI agents
- %ï Token economy ($TRIN + $VSR)
- %ï REST API with 15+ endpoints
- %ï Real-time Socket.IO communication

Next (v5.1)

- %ï More languages: FR, DE, IT, JP, ZH
- %ï Additional n8n workflow templates
- %ï Visual workflow editor in WebOS
- %ï Drag-and-drop agent squad builder
- %ï Real-time agent spawning UI

Future (v6.0)

- %ï Decentralized agent network (p2p)
- %ï Blockchain integration for tokens
- %ï Multi-node cluster support
- %ï Native mobile app (React Native)
- %ï Desktop app (Electron)
- %ï Plugin marketplace for third-party tools
- %ï Visual agent behavior editor

READY FOR THE FUTURE

Trinnity Viseron System v5.0

Presented by: Pedro Costa & Trinnity Hurtado

Architecture: Multi-Agent Superintelligence

Ecosystem: WebOS · Voice · n8n · 5,000+ Agents · Tokens

Contact: pedro@trinnity.com · trinnity@viseron.io

Dashboard: http://localhost:3000